

Reviewer Pack — Minimal Rank of Kähler Curvature Forms vs AC0 Parity Circuit...

Entry 0bbaf7b82545 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 08:07:32 UTC

Minimal Rank of Kähler Curvature Forms vs AC0 Parity Circuit Depth

Entry ID: 0bbaf7b82545 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 08:07:32 UTC

1. Conjecture

Field A (mathematical branch): Complex Geometry (Kähler Manifolds) **Field B** (complexity object): Complexity Theory: AC0 Parity Circuit Complexity

Statement:

[‘For every polynomial-time computable symmetric tensor field T on a Kähler manifold, the minimal rank of its restriction to the unit sphere in any complex linear space is upper-bounded by the maximum depth of an AC0 parity circuit computing T .’, ‘Equivalently, if an AC0 parity circuit can compute T with depth d , then there exists a symmetric tensor field on a Kähler manifold with minimal rank at most 2^d .’]

Rationale (proposer’s reasoning):

[‘Kähler curvature forms encode deep geometric information about the structure of a manifold. If their rank is related to the complexity of AC0 parity circuits, it could provide insights into the intrinsic complexity of certain computations that are hard for classical algorithms.’, ‘This conjecture could potentially expose new structures in the relationship between geometry and computation, leading to novel approaches in algorithm design or complexity theory.’]

Taxonomy category: KÄHLER_CURVATURE_RANK (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 692c0a652383a50d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the mean of the correlation coefficient between the minimal rank of Kähler curvature forms and the AC0 parity circuit depth across all seeds exceeds 0.8, with no seed having a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Kähler manifold" AND "AC0 parity circuit complexity" - "minimal rank Kähler curvature forms" AND "AC0 circuit depth" - "symmetric tensor field T" AND "computing on Kähler manifolds"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
        A[i], A[max_row] = A[max_row], A[i]
        factor = Fraction(A[i][i])
        for j in range(n):
            A[i][j] /= factor
        for k in range(n):
            if k != i:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]

def matrix_multiplication(A, B):
    n = len(A)
    C = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```

def ac0_parity_circuit(T, n):
    if n == 1:
        return T[0][0]
    new_T = [[T[i][j] for j in range(n) if (i + j) % 2 == 0] for i in range(n)]
    depth = 1
    while len(new_T) > 1:
        n_new = len(new_T)
        new_T = [[new_T[i][j] for j in range(n_new) if (i + j) % 2 == 0] for i in range(n_new)]
        depth += 1
    return depth

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    T = [[random.random() for _ in range(n)] for _ in range(n)]
    T = [row[:] for row in zip(*T)] # Ensure symmetry
    rank = sum(1 for row in T if any(row))
    depth = ac0_parity_circuit(T, n)
    return {
        "metric_name": "rank_vs_depth",
        "metric_value": rank,
        "instances_tested": 1,
        "conjecture_holds": rank <= 2**depth,
        "counterexample": "" if rank <= 2**depth else f"Rank {rank} exceeds 2^{depth}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    std_rank = math.sqrt(sum((r["metric_value"] - mean_rank)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_rank} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Rank exceeds 2^depth\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE insufficient evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3224f5b.py", line 75, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3224f5b.py", line 61, in run_trial
    depth = ac0_parity_circuit(T, n)
              ^^^^^^^^^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3224f5b.py", line 51, in ac0_parity_ci
    new_T = [[new_T[i][j] for j in range(n_new) if (i + j) % 2 == 0] for i in range(n_new)]
              ~~~~~^

```

IndexError: list index out of range

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:
 skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:
 The test code crashed before producing data, which means it did not complete its execution to provide a correlation coefficient. | next: Re-run the test code to ensure it completes successfully and produces the required correlation coefficients.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15826
2	propose	ollama_remote	glm4:latest	0	0	14197
3	propose	ollama_remote	glm4:latest	0	0	13687
4	preregistration	ollama_remote	glm4:latest	0	0	9165
5	novelty	ollama_remote	glm4:latest	0	0	8336
6	novelty	ollama_remote	glm4:latest	0	0	9119
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11950
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7828
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8915
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9604
11	judge	ollama_remote	glm4:latest	0	0	11360

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 119989 ms total latency. Provider mix: {'ollama_remote': 11}
(full prompt+response transcripts available in research/audit/0bbaf7b82545.jsonl)

12. Reproducibility

- To reproduce this cycle:
1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
 2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/0bbaf7b82545.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0bbaf7b82545.tar.gz` (if generated)